

TOAP: The Token-Optimized Agent Protocol

A Reference-Minimized, Two-Plane Architecture for Inter-Agent Messaging,
with an Honest Token Accounting and a Multi-Model Pilot

Trilochan Sharma
Independent Researcher
parnishklpo@gmail.com

June 2026

Code and data: <https://github.com/parnish007/TOAP> — Code: MIT — Paper: CC BY 4.0

Abstract

Multi-agent LLM pipelines tend to forward a growing transcript, the source document plus every prior agent’s output, into each downstream prompt; token cost then grows with pipeline depth. An obvious remedy is to pass a *reference* to shared content rather than re-send it. This paper measures that remedy honestly instead of advocating for it, and the results are more nuanced than the pitch. In a controlled experiment ($n=33$ real Claude generations across Haiku, Sonnet, and Opus, three context strategies, accuracy scored by a deterministic and model-independent rubric checker), reference-minimization (TOAP) and a competent summarizing orchestrator *both* cut downstream tokens with no accuracy loss: all 33 samples scored 4/4. But the summarizing baseline saves *more* tokens than TOAP ($\sim 2.5\times$ vs. $\sim 1.9\times$ over the naive baseline). So reference-minimization is not a token win over good prompt engineering. What it offers instead is losslessness (a reference can be re-expanded; a summary cannot), security (provenance and identity travel with the reference), and systematization at the protocol layer. **(3)** A single-sample accuracy regression we initially observed on Haiku **did not reproduce** at $n=5$ and is retracted — a concrete illustration of why $n=1$ agent measurements are unsafe. **(4)** On the compute side, we implement and measure the third materialization, a same-model KV-cache transfer, on seven open models: reusing a transferred prefix cache is a real, near-lossless prefill speedup (35/36 cells byte-identical; up to $\sim 180\times$ at long context) but the cache is thousands of times larger than the text, so it pays off only when agents are co-located or use grouped-query attention — grounding TOAP’s gating policy in measurement rather than assumption. We also separate wire bytes from model tokens and show symbolic “opcodes” save 50% of bytes but only 0–17% of tokens. We describe the protocol architecture (a two-plane wire format; a reference-materialization spectrum) and a Rust reference implementation. *Threats to validity are first-class*: a single model family/tokenizer for the token study, a same-family judge as a secondary check only, and small n . The mechanism is not novel (blackboard systems, A2A artifacts, ADOL); the contribution is the honest, reproducible accounting.

1 Introduction

In agent pipelines and fan-out workflows, an orchestrator typically forwards a growing transcript into each subsequent agent’s prompt. Because LLM cost is paid per prompt token, redundant re-transmission dominates coordination cost. TOAP stores shared content once under an identifier and exchanges references, so each agent retrieves only what it needs.

We try to be conservative about what this buys. The underlying mechanism is old (§2), and “10×” claims that count bytes rather than tokens are misleading (§3). The question worth asking is not whether referencing saves tokens, but how it compares to a competently engineered baseline, and whether small models pay an accuracy price for it. Our central result is a little humbling: a good summarizing orchestrator saves more tokens than reference-minimization at the same accuracy. TOAP therefore has to justify itself on losslessness, security, and systematization rather than on raw token savings, and most of this paper is about establishing where each of those actually matters.

What is measured vs. proposed. The token experiments (§3, §8) use the implemented V1 text protocol and real model generations; the KV-bridge experiment (§8.1) uses the implemented same-model prefix-reuse sidecar. §4.5 states precisely what is implemented today (V1/V2 protocol, the context store with capability-lattice security, the materialization policy, and the KV-bridge sidecar) and what remains future work (durable/distributed store, cross-vendor evaluation, message signing).

Contributions (all framed as a preliminary pilot).

1. A two-plane protocol architecture and a reference-materialization spectrum (§4), with a Rust reference implementation (§5).
2. An honest token accounting separating bytes, tokens, and compute, showing symbolic opcodes are a minor lever (§3).
3. A controlled, $n=33$, bias-free-scored comparison of naive / summarizing / reference strategies, with a retracted single-sample regression (§8).
4. An implemented, seven-model measurement of same-model KV-cache reuse, showing where the compute win justifies the cache-transfer cost (§8.1).
5. A direct answer to “why a protocol rather than a better orchestrator?” (§6).

2 Background and Related Work

Shared-store coordination. Agents coordinating through a common store rather than passing full messages is the *blackboard* model (Hearsay-II [1]); database normalization and pointers are the same principle. TOAP’s context store is a modern restatement, and we claim no novelty for it. **Agent protocols.** MCP [2] (agent–tool) and A2A [3] (agent–agent) are the dominant standards; A2A already references shared content by ID via *artifacts*. Token bloat is recognized: the ADOL Internet-Draft [4] and MCP SEP-1576 target token-efficient messaging (Table 1). **Caching.** Provider prompt caching [5] realizes “send once, reference implicitly” within one provider/session; cross-agent referencing is complementary. **Symbolic comms** [6, 7] cut tokens but risk comprehension; we measure the opcode component at $\leq 17\%$. **Security.** Provenance tracking follows CaMeL [8]. **KV reuse** [9] is crowded and is treated as an optional, constraint-guarded mode.

Protocol	Layer	Wire format	Token efficiency a goal?
MCP (Anthropic)	agent–tool	JSON-RPC 2.0 / HTTP	No
A2A (Google/LF)	agent–agent	JSON-RPC/HTTP (+gRPC/REST)	No
ADOL (IETF draft)	layer above MCP/A2A	JSON \$ref dedup	Yes (content selection)
TOAP (this work)	agent–agent	two-plane text/binary	Yes (encoding, token-aware)

Table 1: Inter-agent protocol landscape (2026). TOAP is adjacent to ADOL/SEP-1576, not ahead of them.

3 Bytes Are Not Tokens

A common mistake is to minimize *bytes* and report the result as token savings. The two are not the same. BPE tokenizers pre-tokenize on punctuation before merging, so a payload dense in punctuation spends close to one token per symbol. Measured with `tiktoken` (cl100k), `SUM(CTX:42)?max_words=150` is 25 bytes and 10 tokens, while the natural-language “Please summarize document 42 in at most 150 words.” is 50 bytes and 12 tokens: half the bytes, but only 17% fewer tokens (Figure 1). Richer payloads save even less. The lesson is that opcode terseness is a minor lever, and the question that actually matters is whether to transmit the *content* at all. We keep three cost planes separate throughout this paper and measure each on its own terms: wire bytes, model-input tokens, and prefill compute.

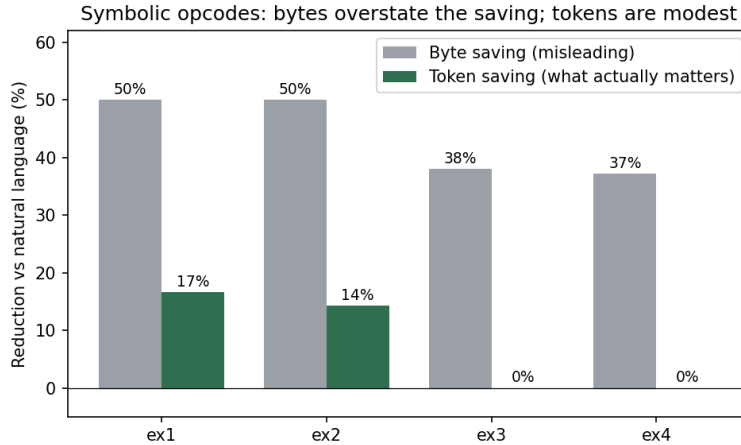


Figure 1: Byte savings overstate token savings for symbolic opcodes (tiktoken cl100k).

4 TOAP Architecture

4.1 System overview

TOAP is a layered, broker-mediated protocol (Figure 3). A message travels *down* the stack on the way out and *up* the stack on the way in; each layer has a single, narrow responsibility. The defining structural choice is that **agents never communicate directly** — every message is mediated by a broker that owns identity, access control, and routing. This is what makes source identity un-spoofable and lets security be enforced in exactly one place.

We describe each layer in turn (§4.1.1–§4.1.6), then the two cross-cutting ideas that distinguish TOAP: the two-plane wire format (§4.2) and the reference materialization spectrum (§4.4).

4.1.1 Agents and the client SDK

Agents (LLMs, tools, or rule-based workers) hold no protocol state of their own; they use a thin async client SDK that performs the SYN/ACK handshake binding identity to the connection, correlates each reply to its request by `msg_id` (so concurrent in-flight requests cannot be confused), and delivers inbound *events* on a channel separate from inbound *requests*. The agent surface is deliberately small: `set_context`, `get_context`, `send_request`, `patch`, `subscribe`.

4.1.2 Protocol layer

The protocol layer turns a message into bytes and back, validating grammar on the way in. V1 is a text frame `TYPE|MSG_ID|TARGET|PAYLOAD` with a function-style payload `OP(args)?k=v&...`; the function form is used precisely because the earlier colon form (`SUM:CTX:42`) is ambiguous when an argument (`CTX:42`) itself contains a colon. The parser is *strict about protocol structure* (delimiter count, payload grammar, context-reference validity) but *never rejects ordinary content* for resembling an attack — SQL, HTML, code, and even “ignore previous instructions” are valid data. V2 adds a compact binary control header (§4.2) that round-trips the identical message model.

4.1.3 Security filter

Before routing, every inbound message passes four checks, in order: (i) *identity* — derived from the authenticated session, never read from the wire; (ii) *ACL* — per-context read/write/delete; (iii) *capability* — does this content’s provenance permit this operation (§4.3); and (iv) *rate/replay* — a per-agent token bucket and per-session duplicate-`msg_id` rejection. A failure at any stage short-circuits to a typed error (`NOPERM/RATE/REPLAY`) and the message is never routed (Figure 2).

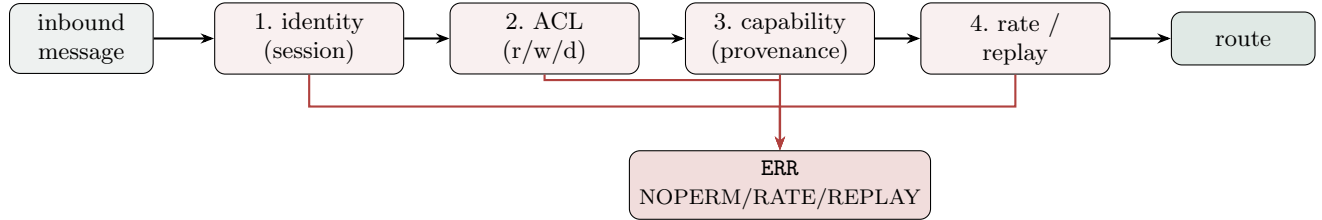


Figure 2: Broker-side security filter. Every inbound message passes four ordered checks; any failure short-circuits to a typed error and is never routed. Identity is derived from the session (never the wire); the capability check refuses, e.g., `EXEC/PAY/DELETE` on user-origin content.

4.1.4 Broker / router

The broker binds one `agent_id` per connection at SYN, routes `REQ` to the target agent, and correlates the returning `RES/ERR` to the original requester by `msg_id` — so a responder addresses its reply to an id, never to a (spoofable) identity, and never learns who asked. A `PATCH (DLT)` to a subscribed context fans out an `EVT` to every subscriber. The broker is, by design, the sole chokepoint; we treat its availability as an explicit threat-model assumption (§9).

4.1.5 Shared context store

Content is stored once under a numeric id and addressed as `CTX:N`. Each entry carries not just bytes but the metadata that makes a reference safe and useful: broker-derived owner, ACL, capability provenance, TTL (with lazy and swept garbage collection), a monotonic version with an append-only field-level delta log, and a subscriber set. This is what lets a reference be *lossless* (the content is recoverable), *secure* (provenance travels with it), and *collaborative* (versioned deltas + events). The current backend is in-memory and single-node; durable/distributed backends are future work.

4.1.6 Transport

The lowest layer frames messages as length-prefixed records over TCP (Tokio). WebSocket and gRPC transports are planned; nothing above the transport layer depends on the choice.

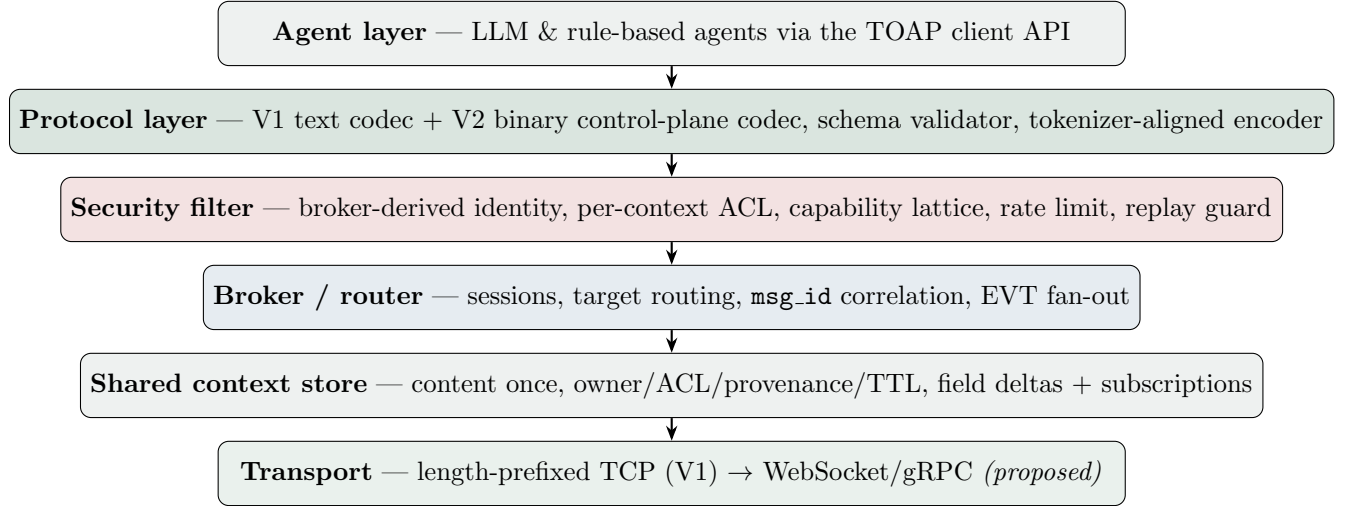


Figure 3: TOAP layered architecture. Identity/trust are broker-owned, never claimed on the wire.

4.2 Two-plane wire format

A single message is read by two parties with opposite cost functions. The *broker* needs ids, session, ACL, capability tags, and nonces — metadata whose cost is *bytes* (bandwidth, parse latency, framing) and which the LLM should never see. The *LLM* needs the operation and its content — text whose cost is *tokens* and which the broker should never tokenize. TOAP encodes these as two planes: a compact binary *control plane* optimized for bytes, and a tokenizer-aligned *semantic plane* optimized for tokens (Figure 4). Because each plane is encoded for its single reader, neither is dominated — which is precisely why “bytes vs. tokens” (§3) stops being a confusion: they are different planes with different objectives.

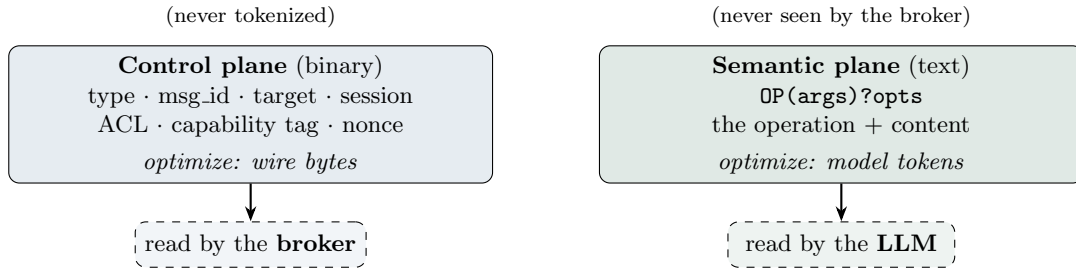


Figure 4: Two-plane message format. Each plane is encoded for its single reader and cost function: the broker reads the byte-optimized control plane and never tokenizes the payload; the LLM reads the tokenizer-aligned semantic plane and never sees the control header.

4.3 Capability-lattice provenance

A boolean “tainted” flag cannot express *what* untrusted content may do. TOAP attaches a provenance to each context: an **Origin** (INTERNAL/EXTERNAL/USER) and the set of **Capability** values that origin permits (read, summarize, transform, classify, execute, email, pay, delete). Trusted internal content permits all; external content permits read-only flows (read/summarize/transform/classify) but no side effects; user-originated content is most restricted. The broker maps each opcode to a capability (**Capability::for_op**) and refuses any operation the provenance forbids — e.g. an EXEC/PAY/DELETE driven by user-origin content returns **NOPERM reason=capability_denied**. Because the tag travels with the reference, taint is preserved across agent-to-agent hops, structurally containing prompt-injection propagation rather than relying on fragile content pattern-matching.

4.4 Reference materialization spectrum

“Send the text,” “send a context id,” and “send a KV-cache pointer” are not three features but three *materializations of one logical reference*, chosen per call by a cost model and degrading gracefully when constraints fail: KV_BRIDGE $\rightarrow$ CTX_REF $\rightarrow$ INLINE. INLINE embeds the bytes (best for tiny/one-shot content); CTX_REF sends the id and the receiver fetches from the store (the dedup win for reused content); KV_BRIDGE points at a reusable KV cache so the receiver skips prefill (valid only for the same model and tokenizer). The same decision — materialize now vs. reference vs. reuse computation — spans both the token plane (resend text vs. send id) and the tensor plane (re-prefill vs. load KV); treating them as one policy is the architecture’s most novel point (Figure 5). Figure 6 shows the token-flow contrast the experiment measures: naive prompts grow with pipeline depth while references stay flat, because the store holds content once and only ids travel.

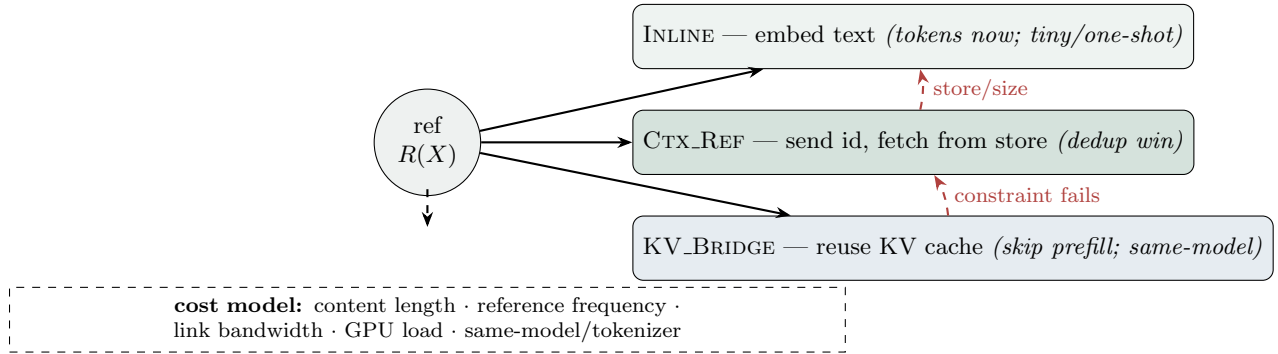


Figure 5: Reference materialization spectrum: one logical reference, three materializations chosen per call by a cost model, with graceful degradation KV_BRIDGE $\rightarrow$ CTX_REF $\rightarrow$ INLINE when constraints fail. The same decision spans the token plane (resend vs. id) and the tensor plane (re-prefill vs. load KV).

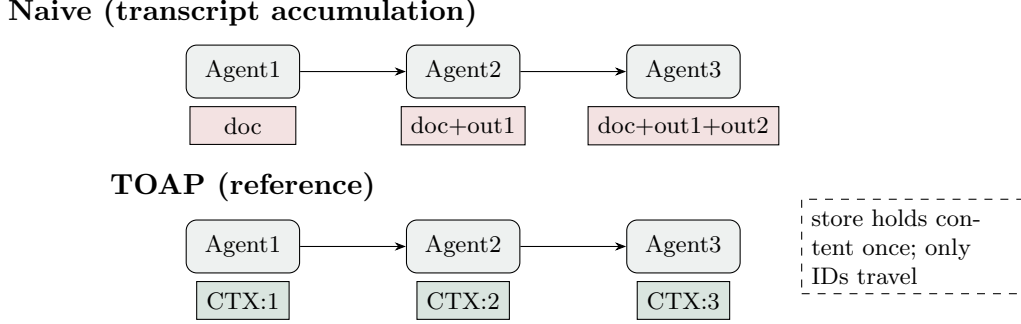


Figure 6: Token-flow contrast: naive prompts grow with depth; references stay flat.

4.5 Implemented vs. proposed

Implemented today (28 passing unit + integration tests): the V1 text protocol and parser; the **V2 binary control plane** (compact binary control header + text semantic payload — the two-plane design, with round-trip tests); the shared context store with per-context ACL, TTL, field-level deltas, and subscriptions; a **capability-lattice** provenance model ($\text{Origin} \in \{\text{Internal}, \text{External}, \text{User}\}$, each with an allowed-capability set) the broker enforces structurally (user-/external-origin content cannot flow into EXEC/PAY/DELETE); an **in-band cache coordinate** with a stable-prefix || volatile-suffix layout helper; the **materialization policy with graceful fallback** (KV_BRIDGE $\rightarrow$ CTX_REF $\rightarrow$ INLINE) behind a KvTransport trait; broker-derived identity, token-bucket rate limiting, per-session replay rejection, `msg_id` correlation; and an MCP frontend.

A minimal **KV-cache transport** is now implemented as a same-model, same-tokenizer *prefix-reuse* sidecar (`benchmark/kv_bridge`): it extracts a shared context’s KV, serializes/transfers it, and lets the receiver prefill only its query suffix, with a benchmark that measures prefill-latency saved, KV-byte-size vs. text-size, and exact greedy-output correctness. The Rust side owns the policy and fallback (`RegistryKvTransport` enforces the same-model constraint); the sidecar owns the tensor transport. We deliberately restrict to prefix reuse (absolute positions preserved) and do *not* splice caches across positions. Cross-vendor evaluation remains future work.

5 Implementation

TOAP is a Rust workspace. `toap-core` defines the message model, a strict recursive-descent parser, and the encoder; it deliberately rejects malformed *protocol* syntax but never rejects ordinary content for “looking dangerous” (SQL/HTML/prompt-like text are data). `toap-context` implements the in-memory store: numeric context IDs, an ACL grammar (`agent:rw`), a *capability-lattice* provenance model ($\text{Origin} \times \text{allowed Capability set}$, with `Capability::for_op` mapping opcodes to capabilities), TTL with lazy + sweep GC, a bounded per-context delta log, the **Reference** materialization enum with a `materialize_with_fallback` policy behind a KvTransport trait, and a `cache_layout` helper for stable-prefix ordering. `toap-core` also provides a V2 binary control-plane codec (`encode_v2/decode_v2`) that round-trips the V1 message model. `toap-security` provides a token-bucket rate limiter and a per-session replay guard; the broker enforces the capability lattice structurally, refusing e.g. an EXEC/PAY/DELETE against user- or external-origin context. `toap-broker` is an asynchronous (Tokio) TCP server with length-prefixed framing; identity is bound to the connection at SYN, and a responder addresses replies by the original `msg_id` so it never learns the requester (preventing source spoofing). `toap-client` is an async SDK; `toap-mcp` is the MCP frontend.

Testing is unit *and* integration: parser round-trip/validation and store/ACL/TTL/delta unit tests, plus end-to-end integration tests in which **real client processes connect to the broker over TCP** and exchange requests — the broker performs actual routing and store operations (not mocked). Security integration tests assert that an unauthorized read is denied, a restricted op on tainted context is refused, and a subscriber receives a delta event. A live multi-process demo (broker + summarizer + classifier + orchestrator) exercises fan-out and merge. Coordination-frame sizes reported below are measured on the actual encoder output.

6 Why a Protocol, Not Just a Better Orchestrator?

The sharpest objection is: “just write an orchestrator that summarizes before forwarding.” Our experiment confirms a summarizing orchestrator is in fact *more* token-efficient than referencing (§8). So the case for a protocol does not rest on token savings. It rests on three properties a prompt-engineering pattern does not provide: **(i) Losslessness** — a reference can be re-expanded to the full content on demand; a summary discards information irreversibly, which is safe only until a downstream task needs a dropped detail. **(ii) Security travels with the reference** — provenance/taint and broker-derived identity are enforced structurally, so a compromised agent cannot launder untrusted content or spoof a source; a summarizing prompt template offers none of this. **(iii) Systematization** — the behavior is uniform across heterogeneous agents and languages and composes with deduplication (send-once/refer-many) and provider caching, rather than being re-implemented per application. TOAP should be adopted for these reasons, or not at all.

7 Experimental Method

Instrument. Each agent call is an isolated Claude subagent at an explicit model (Haiku/Sonnet/Opus), tool use disabled; outputs are genuine generations. **Controlled design.** To get clean variance we fix one canonical upstream (one document, one extracted-facts text, and one analysis containing all four rubric themes) so the only variables are {model, context strategy, sample}. Three strategies for the final *writer* stage: *naive* (document + facts + analysis), *summary* (a short lossy summary of the analysis, i.e. a competent summarizing orchestrator), and *TOAP* (the analysis only, i.e. the distilled slice passed by reference). We draw $n=5$ samples per cell for Haiku and $n=3$ for Sonnet/Opus ($n=33$ total). **Accuracy (bias-free).** The primary accuracy measure is a *deterministic* rubric checker: each of four required actions is detected by keyword sets, with no model in the loop, so it is immune to same-family judge bias and exactly reproducible. (A model judge was used only in an earlier single-sample run and is not relied upon here.) **Tokens.** Counted with `tiktoken` (cl100k) on the exact prompts and verbatim outputs. **Reproducibility.** All prompts, verbatim outputs, and the scorer are committed.

Reviewer-driven changes from the first draft. (1) $n=1$ replaced by $n=33$ with ranges; (2) a *strong* summarizing baseline added alongside the naive one; (3) a deterministic, model-independent scorer replaces reliance on a same-family judge; (4) the architecture now flags implemented vs. proposed; (5) all empirical claims framed as a preliminary pilot.

8 Results

Accuracy parity; the regression retracted. All 33 samples scored 4/4 on the deterministic rubric (Figure 8); coverage variance was zero. A 4/4→3/4 Haiku regression seen in an earlier *single-*

sample run **did not reproduce** at $n=5$ with a fixed upstream; we attribute it to single-sample noise and/or propagation of a weaker upstream analysis, and retract it.

Tokens: the summarizing baseline beats TOAP. Table 2 and Figure 7 give total writer tokens. Against the naive baseline, TOAP cuts tokens by $1.88\text{--}1.93\times$ (mean $\sim 1.91\times$); the summarizing baseline cuts them by $2.39\text{--}2.62\times$ (mean $\sim 2.54\times$). Both hold 4/4 accuracy. Reference-passing, in other words, is not the most token-efficient strategy; a competent summarizer is. We state this plainly because it is the result most likely to be glossed over, and because it reframes what the rest of the protocol is for.

Model	Strategy	n	Accuracy (/4)	Total tokens (mean [min-max])	Reduction vs. naive
Haiku	naive	5	4.0	505 [491-520]	—
Haiku	summary	5	4.0	211 [193-219]	$2.39\times$
Haiku	TOAP	5	4.0	268 [258-279]	$1.88\times$
Sonnet	naive	3	4.0	512 [499-523]	—
Sonnet	summary	3	4.0	195 [190-204]	$2.62\times$
Sonnet	TOAP	3	4.0	266 [265-267]	$1.93\times$
Opus	naive	3	4.0	504 [487-519]	—
Opus	summary	3	4.0	194 [192-196]	$2.60\times$
Opus	TOAP	3	4.0	263 [253-271]	$1.92\times$

Table 2: Controlled writer experiment ($n=33$). Accuracy by deterministic rubric (bias-free). Tokens via tiktoken. TOAP < naive, but summary < TOAP.

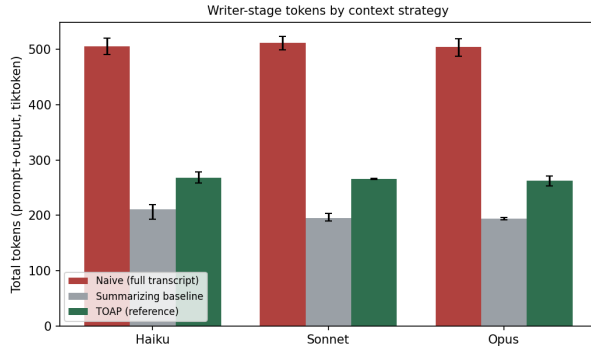


Figure 7: Writer tokens by strategy (error = min-max). Summary < TOAP < naive.

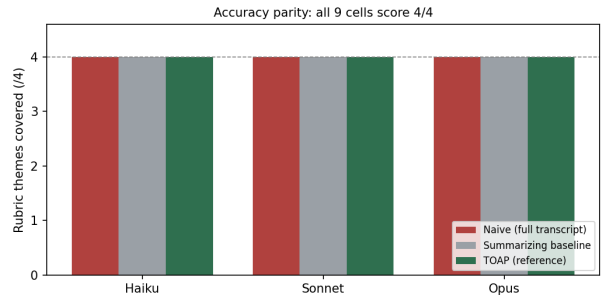


Figure 8: Accuracy parity: all nine cells 4/4 (the $n=1$ regression did not reproduce).

Where referencing does win: multi-hop and coordination. The writer experiment is a single hop. In a multi-hop pipeline, referencing avoids re-summarizing/re-sending at *every* hop, and the store’s send-once/refer-many compounds; an exploratory three-stage run showed $1.65\text{--}1.82\times$ downstream reductions. Separately, TOAP coordination frames are $\sim 17\times$ smaller in tokens than JSON-RPC equivalents (model-independent, exactly reproducible). These are the regimes where a reference protocol, rather than per-hop summarization, is the natural fit — and where losslessness matters most.

8.1 KV-bridge: prefill reuse for same-model agents

The third materialization, KV_BRIDGE, transfers a shared context’s key/value cache so the receiver prefills only its query suffix. We evaluate the same-model, same-tokenizer prefix-reuse case (absolute

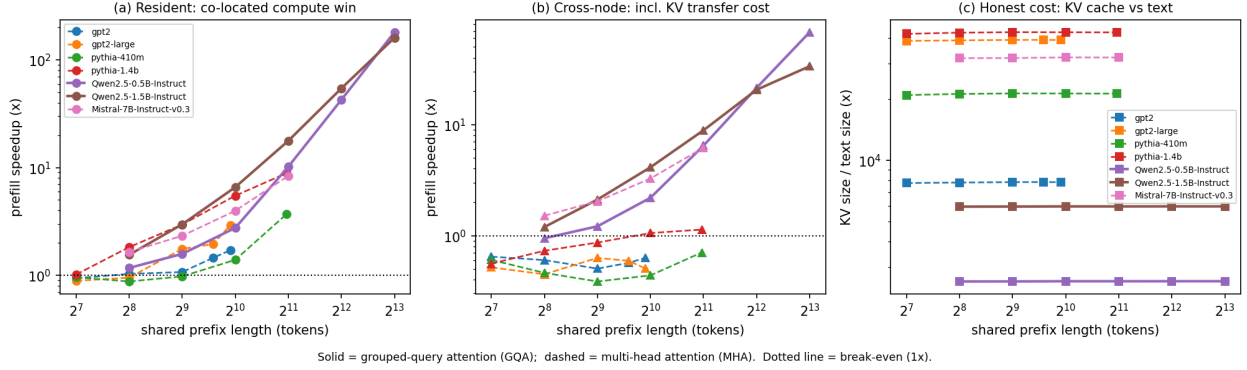


Figure 9: KV-bridge prefill reuse across seven models on an NVIDIA T4 (36 cells, log scales). (a) Resident speedup grows with prefix length and is dramatic for GQA models (solid), reaching $\sim 180\times$ at 8k tokens. (b) With KV transfer charged, GQA stays above break-even while MHA (dashed) mostly does not. (c) The KV cache is thousands of times larger than the text, and far larger for MHA than GQA — the reason transfer cost flips the verdict by architecture.

Model	Attn	Max prefix	KV/text	Resident	Cross-node	Lossless
GPT-2 (124M)	MHA	960	7.8k	1.7 $\times$	0.6 $\times$	4/5
GPT-2-large (774M)	MHA	960	39k	2.9 $\times$	0.5 $\times$	5/5
Pythia-410M	MHA	2000	21k	3.7 $\times$	0.7 $\times$	5/5
Pythia-1.4B	MHA	2000	43k	8.9 $\times$	1.1 $\times$	5/5
Mistral-7B (4-bit)	MHA	2048	32k	8.4 $\times$	6.2 $\times$	4/4
Qwen2.5-0.5B	GQA	8192	2.5k	181$\times$	68$\times$	6/6
Qwen2.5-1.5B	GQA	8192	5.9k	161$\times$	34$\times$	6/6

Table 3: KV-bridge at each model’s longest tested prefix. Resident = co-located compute win; cross-node charges KV transfer. KV/text is the cache size relative to the text it replaces. Lossless counts cells with byte-identical greedy output (35/36 overall).

positions preserved; no cross-position splicing) on an NVIDIA T4 across seven open models that span attention designs: multi-head attention (GPT-2, GPT-2-large, Pythia-410M/1.4B), grouped-query attention with very few KV heads (Qwen2.5-0.5B/1.5B), and a 4-bit 7B model (Mistral-7B). Timing is warmed up and CUDA-event-timed, and isolates *prefill* (decode excluded). Each cell reports two speedups: *resident* (= recompute prefill / query-prefill on an already-resident KV — the co-located / shared-broker case, the pure compute win) and *cross-node* (additionally charging the serialize/transfer of the KV cache). Correctness is an exact greedy-token match against recompute. Three things come out of the run, and they match the hypothesis that motivated the gating policy. First, reuse is a *real* compute saving: once the shared prefix passes a crossover of a few hundred tokens, the resident speedup climbs steeply — to roughly $180\times$ for the half-billion- parameter Qwen at an 8k prefix, because recompute pays $O(n^2)$ attention over the whole prefix while the bridge pays only for the short query. Below the crossover the two are within noise ($\sim 0.9\text{--}1.1\times$), which is exactly why the policy does not bridge tiny contexts. Second, the saving is essentially *lossless*: 35 of 36 cells produced byte-identical greedy output. The single exception (GPT-2 at a 512-token prefix) is a one-token divergence from float16 rounding — prefilling the whole sequence and prefilling the query on top of a reused cache accumulate rounding slightly differently — and would disappear in float32; we report it rather than hide it. Third, and most decisive for the architecture, the KV cache is *thousands* of times larger than the text it stands in for (Table 3), and the ratio depends sharply

on attention design: grouped-query models (Qwen, two KV heads) carry an order of magnitude less cache than multi-head models (GPT-2-large, twenty heads). The consequence shows up in the cross-node column: once the transfer cost of moving that cache is charged, GQA models stay comfortably above break-even (Qwen reaches $68\times$) while most MHA models fall below it. In other words, KV-bridge is worth it exactly when the producer and consumer are co-located *or* the model uses grouped-query attention — which is precisely the condition `RegistryKvTransport` encodes, falling back to `CTX_REF` otherwise. The measurement grounds the policy rather than assuming it.

9 Discussion and Limitations

The honest synthesis: *naive transcript accumulation is wasteful; both summarization and referencing fix it at equal accuracy here; summarization is cheaper on tokens but lossy; referencing is slightly costlier but lossless, secure, and systematic* (§6). The retracted Haiku regression is itself a result: **single-call agent measurements are unsafe**, and apparent small-model effects must be checked with replication.

We state threats plainly and treat them as first-class. **Single model family / one tokenizer:** Haiku/Sonnet/Opus differ by scale, not architecture; token claims are tokenizer-dependent, so cross-vendor (GPT/Gemini/Llama) replication is required and untested. **Small n and a single task:** $n=33$ on one incident scenario with a fixed upstream; broader datasets are needed. **Saturated accuracy:** the four-theme task was easy enough that all strategies reached 4/4, so it does not stress the losslessness advantage — a harder task that needs a detail a summary would drop is exactly the missing experiment. **Scorer:** keyword coverage is bias-free but coarse; it can miss paraphrase. **Judge:** any model judge here is same-family and is used only as a discarded secondary check. **KV-bridge scope:** the speedups are microbenchmarks of one prefill step on a single GPU (T4), against full recompute — not against provider prefix-caching, which is the production alternative and which we do not claim to beat; “transfer” is local serialize/deserialize, not a real RDMA hop; and we restrict to same-model prefix reuse with no cross-position splicing. The float16 token divergence in one of 36 cells is a reminder that “lossless” here means token-identical in practice, not bit-exact in theory. **Implementation scope:** the V2 binary control plane, capability lattice, cache-coordinate layout, materialization-with-fallback, and the KV-bridge sidecar are implemented and tested; what remains future work is a durable/distributed context store, message signing / cross-reconnect replay nonces, and cross-vendor (non-Claude) replication of the token study.

On novelty. We claim none for the mechanism. The contribution is the controlled, bias-free, reproducible accounting — including the finding that a summarizing baseline beats referencing on tokens — and an architecture that makes the byte/token planes and the materialization choice explicit.

10 Conclusion

Two measurements, on two cost planes, point the same way. On tokens, reference-minimization removes the waste of transcript accumulation at no measured accuracy cost — but it is not the cheapest option, a competent summarizing orchestrator saves more, so TOAP earns referencing’s place through losslessness, in-band security, and systematization rather than raw token count. On compute, transferring a same-model KV cache is a real, near-lossless prefill speedup that grows to two orders of magnitude on long contexts, yet the cache is so much larger than the text that it only pays off co-located or under grouped-query attention. The common thread is that no single way of sharing context dominates: the right choice depends on length, architecture, co-location,

and whether lossiness is acceptable — which is the case for making materialization an adaptive, cost-model-driven decision rather than a fixed mechanism. We release all artifacts and call for cross-vendor, large- n , harder-task replication — especially a task that stresses the lossless-versus-lossy distinction, and a comparison of KV-bridge against provider prefix-caching — before any general claim.

References

- [1] L. Erman, F. Hayes-Roth, V. Lesser, D. Reddy. The Hearsay-II Speech-Understanding System (blackboard model). *ACM Computing Surveys* 12(2), 1980.
- [2] Anthropic. Model Context Protocol specification, 2024–2026. <https://modelcontextprotocol.io>.
- [3] Google / Linux Foundation. Agent2Agent (A2A) Protocol v0.3, 2025–2026. <https://a2a-protocol.org>.
- [4] Z. Chang, J. Li, Z. Cao. A Token-Efficient Data Layer for Agents (ADOL). IETF Internet-Draft `draft-chang-agent-token-efficient-01`, Dec. 2025.
- [5] Anthropic, OpenAI, Google. Prompt caching documentation, 2024–2026.
- [6] Z. Wang et al. Talk Structurally, Act Hierarchically (TalkHier). arXiv:2502.11098, 2025.
- [7] CodeAgents: pseudocode-codified multi-agent messages. arXiv:2507.03254, 2025.
- [8] Google DeepMind. Defeating Prompt Injections by Design (CaMeL). arXiv:2503.18813, 2025.
- [9] KVCOMM: Online Cross-context KV-cache Communication. arXiv:2510.12872, 2025.

A Reproducibility

Artifacts: `benchmark/ab_study/` (harness, scenarios, the $n=33$ run records `runs/writer_runs.json`, deterministic scorer `writer_experiment.py`, analysis `compute_writer.py`); `paper/figures/` (figure scripts); `paper/METHODS.md` and `paper/REVIEW_RESPONSE.md` (method log and the reviewer-fix trail). The Rust implementation is in `crates/` (28 unit + integration tests, incl. real-TCP end-to-end, V2 binary round-trip, capability-lattice flow, cache layout, and KV-bridge fallback). Token reductions recompute exactly from the committed records via `compute_writer.py`.